

Model Card — ColdTrack AI

Versi: v2 (hibrida) · **Tanggal:** 7 Agustus 2026 · **Penanggung jawab:** R2 (AI Model Engineer)

Dokumen ini menjelaskan model yang dipakai ColdTrack AI, cara pengujiannya, dan — sama pentingnya — **di mana model ini tidak bisa diandalkan**. Seluruh angka di sini berasal dari split **test**, bagian data yang tidak pernah dipakai untuk mengambil keputusan apa pun selama pengembangan.

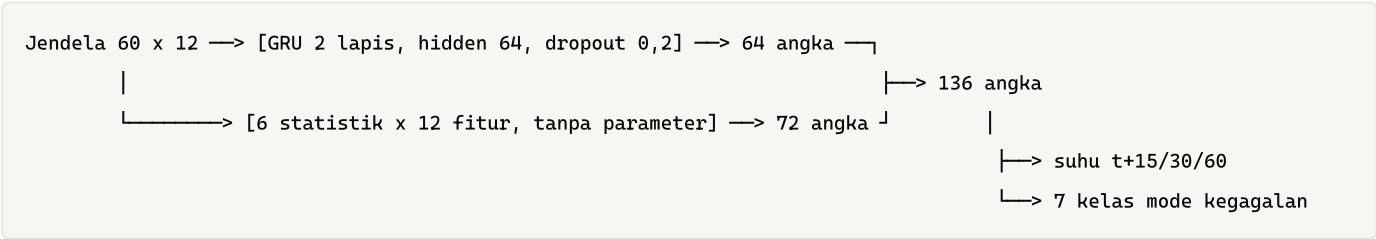
Model Details

Sistem ini memakai **dua model** yang berbagi kontrak masukan identik.

	coldtrack.onnx	coldtrack_ttb.onnx
Jenis	GRU 2 lapis (hidden 64) + statistik ringkasan jendela	XGBoost, 300 pohon
Keluaran	prediksi suhu t+15/30/60, probabilitas 7 mode kegagalan	Time-to-Breach (menit)
Ukuran	169 KB	950 KB
Latensi CPU (batch 1)	1,1 ms	0,09 ms
Parameter / pohon	41.443 parameter	300 pohon

Masukan (sama untuk keduanya): tensor `float32` berbentuk `[batch, 60, 12]` — jendela 60 menit berturut-turut, 12 fitur per menit sesuai `docs/feature_schema.md`. Nilai dikirim **mentah**; normalisasi dan perhitungan statistik agregat dibungkus di dalam grafik ONNX masing-masing model, sehingga backend tidak perlu mengulang langkah prapemrosesan apa pun.

Arsitektur GRU



Jalur statistik ringkasan (rata-rata, simpangan baku, min, maks, nilai terakhir, tren) ditambahkan setelah diagnosis menunjukkan GRU menghabiskan kapasitas untuk mempelajari besaran yang sebenarnya dapat dihitung langsung. Setelah ditambahkan, Macro F1 naik dari 0,440 ke 0,598 di split validasi dengan tambahan hanya 792 parameter.

Pemetaan 9 mode anomali menjadi 7 kelas

Dua pasang mode digabung karena tanda tangan termalnya nyaris tidak dapat dibedakan dari jendela 60 menit:

Kelas keluaran	Berasal dari	Alasan penggabungan
degradasi_bertahap	A2 (kompresor melemah) + A4 (kebocoran refrigeran)	keduanya menghasilkan drift naik yang lambat
masalah_sensor	A5 (sensor macet) + A6 (sensor berderau)	keduanya kegagalan sensor, bukan kegagalan kargo

Kelas keluaran	Berasal dari	Alasan penggabungan
A0 , A1 , A3 , A7 , A8	tidak digabung	tanda tangannya khas

Intended Use

Untuk apa model ini dibuat. Memberi peringatan dini pada operator armada rantai dingin bahwa suhu kargo sedang menuju keluar dari pita aman, sehingga masih ada waktu bertindak — menepi, memanggil teknisi, atau mengalihkan muatan.

Yang bukan tujuannya.

- **Bukan penentu keputusan otomatis.** Keluaran model adalah masukan bagi operator manusia dan rule engine, bukan pengganti keduanya.
- **Bukan prediktor umur simpan.** Model memprediksi kapan suhu keluar ambang, **bukan** kapan produk membusuk. Keduanya hal berbeda; ColdTrack AI tidak memiliki data pembusukan.
- **Bukan sertifikat kepatuhan.** Tidak menggantikan pencatatan suhu resmi yang disyaratkan CDOB atau HACCP.
- **Bukan hitung mundur jarak jauh.** Lihat bagian keterbatasan Time-to-Breach di bawah.

Training Data

Sumber utama: `data/processed/v4_seed1000_700trips.parquet` — 700 perjalanan sintetik, 251.073 baris, resolusi 1 menit, dibangun R1 dengan simulator termal berbasis hukum pendinginan Newton. Rincian sumber, kalibrasi, dan validasi sim-to-real ada di `docs/dataset_card.md`.

Data dipecah menjadi jendela geser 60 menit, dibagi **per** `trip_id` (bukan per baris) agar tidak ada perjalanan yang sama muncul di dua split:

Split	Jendela	Peran
train	77.703	melatih bobot
val	16.414	memilih arsitektur dan hiperparameter
test	16.244	hanya untuk pelaporan akhir

Model dilatih dari nol, bukan dari backbone pretrained

Rencana awal memakai transfer learning dua tahap: prapelatihan pada data suhu IoT publik, lalu fine-tuning pada domain rantai dingin. Rencana itu **diuji dan ditinggalkan** karena tidak terbukti memberi manfaat — lihat Ablation B di bawah.

Skrip prapelatihan (`ml/pretrain.py`) dan korpusnya tetap dipertahankan di repositori sebagai bukti eksperimen, tetapi tidak berada di jalur produksi.

Evaluation

Seluruh angka berikut dari split **test**.

Terhadap target resmi

Head	Metrik	Hasil	Target	Status
Forecast suhu	MAE @ t+30	0,198 °C	< 0,8 °C	tercapai

Head	Metrik	Hasil	Target	Status
Mode kegagalan	Macro F1	0,581	> 0,80	belum
Time-to-Breach	MAE (TTB $\leq$ 30 menit)	7,08 menit	< 8 menit	tercapai
Deteksi anomali	PR-AUC	0,711	> 0,85	belum

Dua dari empat target tercapai. MAE forecast pada horizon lain: 0,152 °C (t+15) dan 0,237 °C (t+60).

Perbandingan dengan baseline

Untuk menjawab pertanyaan "mengapa perlu deep learning", tiga baseline diuji pada data yang sama:

Metrik	GRU fusion	XGBoost	Regresi linear	Isolation Forest
Forecast t+30 (°C)	0,198	0,189	0,338	—
Macro F1	0,581	0,664	—	—
Akurasi	0,813	0,871	—	—
PR-AUC anomali	0,711	0,753	—	0,371
TTB $\leq$ 30 menit (menit)	10,8	7,1	—	—

XGBoost mengungguli GRU pada seluruh metrik. Temuan ini dilaporkan apa adanya, dan menjadi dasar keputusan memakai XGBoost untuk Time-to-Breach.

GRU tetap dipakai untuk forecast dan klasifikasi dengan pertimbangan: selisih pada forecast tipis (0,198 vs 0,189, keduanya jauh melampaui target), pada klasifikasi kedua model sama-sama belum mencapai target sehingga penggantian tidak mengubah status papan skor, dan satu model GRU melayani dua keluaran sekaligus dengan ukuran 169 KB.

Akurasi per kelas kegagalan

Kelas	Recall	Jumlah sampel
A3 — kegagalan reefer total	100,0 %	379
A0 — sehat	92,1 %	12.546
masalah_sensor (A5+A6)	66,9 %	872
A1 — pintu terbuka lama	58,9 %	321
A7 — kejut suhu ambien	44,1 %	642
A8 — prapendinginan buruk	20,1 %	651
degradasi_bertahap (A2+A4)	10,6 %	833

Pola ini masuk akal secara fisika: A3 (reefer mati total) menghasilkan kenaikan monoton yang khas dan selalu terdeteksi, sementara **degradasi_bertahap** justru mode yang paling sulit — perubahannya lambat dan dalam jendela 60 menit nyaris tidak terbedakan dari kondisi sehat. Ironisnya, itulah mode yang paling berbahaya di dunia nyata karena juga luput dari pengamatan manusia.

Ablation study

A. Multi-task vs single-task. Arsitektur "satu backbone, tiga kepala" menahan performa:

Tugas	Multi-task	Single-task
Forecast t+30	0,244	0,203
Macro F1	0,440	0,613
Time-to-Breach	22,94	23,62

Kerugian ini kemudian **hampir sepenuhnya hilang** setelah jalur statistik ringkasan ditambahkan (Macro F1 multi-task naik ke 0,598, mendekati 0,613 milik single-task).

B. Fine-tuning vs dilatih dari nol. Pretraining tidak memberi manfaat — bahkan merugikan:

	Fine-tuned	Dari nol
Loss latih akhir	2,99	3,58
Loss validasi akhir	4,18	2,78

Varian pretrained mencapai loss latih lebih rendah tetapi loss validasi lebih tinggi — pola overfitting yang khas. Grafiknya ada di `ml/reports/loss_curves.png`.

Penyebabnya dapat dijelaskan. Korpus prapelatihan (Intel Berkeley Lab, 1,3 juta jendela) hanya mampu mengisi **4 dari 12 fitur** — data sensor ruangan tidak memiliki informasi pintu, reefer, kecepatan, maupun radiasi matahari. Delapan fitur sisanya bernilai nol, sehingga bobot GRU terspesialisasi pada distribusi masukan yang timpang, dan spesialisasi itu justru harus dilupakan saat fine-tuning.

Upaya perbaikan sudah dilakukan sebelum menyimpulkan: korpus prapelatihan diganti dari 6.113 jendela (IoT India, 3 fitur terisi) menjadi 1,3 juta jendela (Intel Lab, 4 fitur terisi) — 207 kali lebih besar. Hasilnya tetap sama.

Limitations & Ethical Considerations

1. Time-to-Breach hanya andal untuk peringatan jangka pendek

TTB sebenarnya	MAE
≤ 10 menit	3,30 menit
≤ 30 menit	7,08 menit
keseluruhan	52,00 menit

Model **andal sebagai alarm "sementar lagi jebol"**, **tidak andal sebagai hitung mundur jarak jauh**. Ini keterbatasan fisik, bukan kekurangan implementasi: memperkirakan kejadian lima jam ke depan dari jendela 60 menit berada di luar jangkauan informasi yang tersedia.

Implikasi produk: tampilkan angka TTB hanya bila nilainya di bawah ~30 menit. Di atas itu, tampilkan status risiko tanpa angka spesifik agar tidak memberi kesan presisi yang tidak dimiliki model.

2. Keluaran TTB tidak terdefinisi untuk kondisi sehat

Model TTB dilatih hanya pada jendela yang benar-benar menuju breach; jendela sehat di-mask dari loss. Akibatnya, model **tetap mengeluarkan angka** saat kondisi sehat, dan angka itu tidak bermakna — pengujian pada 2.000 jendela sehat menghasilkan median **17 menit**, dan **tidak satu pun** bernilai besar yang dapat dipakai backend sebagai penanda "aman".

Angka TTB wajib disembunyikan bila `failure_prob` menunjuk ke kelas `A0`. Ini bukan saran, melainkan syarat kebenaran keluaran.

3. Mode degradasi bertahap hampir tidak terdeteksi

Recall 10,6 %. Sistem **tidak boleh dipromosikan sebagai pendeteksi kompresor melemah atau kebocoran refrigeran**. Kegagalan jenis ini akan lolos pada sebagian besar kasus.

4. Dilatih sepenuhnya pada data sintetik

Tidak ada telemetri truk berpendingin nyata dalam data latih, karena dataset publik semacam itu tidak tersedia. Simulator dikalibrasi terhadap dataset IoT publik dan standar suhu resmi (WHO, BPOM No. 6/2020, Codex Alimentarius, SNI), dan divalidasi lewat uji KS serta ACF — tetapi **belum pernah divalidasi terhadap perjalanan truk sungguhan**. Performa di lapangan berpotensi berbeda, dan uji lapangan wajib dilakukan sebelum penggunaan operasional.

5. Split test tidak seimbang untuk TTB

Split test hanya memuat 10,6 % jendela breach, sementara val memuat 22,6 %, dengan median TTB 63 vs 10 menit. Pembagian data distratifikasi per `failure_mode` tetapi tidak per kejadian breach, sehingga estimasi TTB antar split kurang sebanding. Angka test tetap dipakai karena lebih konservatif.

6. Risiko false negative lebih berat daripada false positive

Alarm palsu merugikan waktu operator; kegagalan terdeteksi berarti muatan rusak — pada kasus vaksin, berpotensi membahayakan penerima. Ambang keputusan pada rule engine sebaiknya condong ke sisi waspada, dan sistem tidak boleh dijadikan satu-satunya lapis pengaman.

7. Model bersifat statis

Bobot dibekukan saat ekspor. Model tidak belajar dari data baru saat berjalan, sesuai ketentuan lomba mengenai parameter statis. Setiap pembaruan menuntut pelatihan ulang dan verifikasi ulang.

Reproduksi

```
python -m ml.preprocess.build_windows      # jendela dari dataset v4
python -m ml.finetune                      # latih GRU (~35 menit CPU)
python -m ml.export_onnx                   # ekspor coldtrack.onnx + labels.json
python -m ml.export_ttb_xgboost            # ekspor coldtrack_ttb.onnx (~1 menit)
python -m ml.make_reports                  # metrics.json + grafik
python -m ml.baselines                     # baseline pembanding
python -m ml.ablation                      # ablation A & B (~2,5 jam)
```

Seluruh skrip memakai `seed = 42`. Bobot model (`*.pt`) tidak disertakan di repositori; jalankan `ml.finetune` untuk membangunnya kembali.

Berkas terkait: `ml/reports/metrics.json` · `baseline_metrics.json` · `ablation_results.json` · `loss_curves.png` · `confusion_matrix.png` · `ttb_by_horizon.png` · `docs/dataset_card.md` · `docs/feature_schema.md`